

Summary of Pre-Training at Scale

Saitej Velaga (saitejv), Jack McClure (jackmccl), Angela Li (anqili)

Lecture Date: September 16, 2026

1. Research Direction of the Field

The research area of this lecture is **efficient distributed training of very large language models**, particularly how to divide computation, parameters, data, and communication across thousands of GPUs while keeping those GPUs busy. As model sizes and context lengths increase, a single accelerator cannot hold the entire model or finish training in a reasonable amount of time. The difficulty is therefore no longer simply finding enough GPUs: different forms of parallelism introduce communication, memory, synchronization, and load-balancing costs that can erase the benefits of additional hardware.

Early large-scale training systems addressed different pieces of this problem separately. **Tensor parallelism (TP)**, popularized by Megatron-LM, divides individual Transformer operators across GPUs, while **pipeline parallelism (PP)** divides layers into stages and pipelines microbatches through them. Megatron-LM showed that combining tensor, pipeline, and data parallelism could train trillion-parameter models across thousands of GPUs, and introduced interleaved 1F1B scheduling to reduce pipeline bubbles. In parallel, ZeRO/FSDP reduced memory redundancy by sharding parameters, gradients, and optimizer states. As clusters grew beyond 10,000 GPUs, work such as MegaScale shifted the focus toward full-system co-design: communication-computation overlap, network tuning, operator optimization, observability, and straggler handling all became necessary to maintain utilization at production scale. MegaScale, for example, reported 55.2% MFU for a 175B model on 12,288 GPUs.

The two lecture papers represent successive stages of this evolution. **Scaling Llama 3 Training** shows how multidimensional parallelism became a practical production system rather than a collection of independent techniques. Llama 3 combines FSDP, TP, PP, and context parallelism (CP), choosing their dimensions according to batch size, model memory, sequence length, and network topology. It additionally modifies pipeline scheduling to support changing batch sizes and heterogeneous multimodal layers, and uses CP to make 128K-context training feasible. The important shift is from asking *which parallelism method should we use?* to asking *how should multiple parallelisms be co-designed with the model and hardware?*

Tessera addresses the next complication: modern MoE models are no longer stacks of nearly identical Transformer blocks. Qwen3-Next, for example, mixes Gated DeltaNet and full-attention layers with sparse MoE blocks, so different pipeline stages have very different compute/communication behavior. As a result, balancing stages using their serial execution times can actually produce imbalance after communication overlap is applied. Tessera therefore jointly considers overlap scheduling and partitioning and additionally adapts to stochastic MoE routing at runtime. This suggests a broader direction for the field: **parallelism policies are becoming increasingly workload-aware and adaptive rather than static**. Recent MoE systems likewise

explore heterogeneous mappings of attention and expert layers rather than forcing one parallel configuration on the entire model. Because Tessera appeared only in OSDI 2026, there is not yet a substantial body of direct follow-up work.

2. Lecture Papers

Paper 1: *Scaling Llama 3 Training with Efficient Parallelism Strategies*

What is the problem of this paper?

Llama 3 405B required months of pre-training on as many as 16,384 H100 GPUs. At this scale, simply adding GPUs does not guarantee faster training because the fixed global token budget limits data parallelism, while TP, PP, FSDP, and CP each impose different communication and memory costs. The workload also changes over training: sequence lengths grow from 8K to 131K, global batch sizes change, and multimodal training introduces an image encoder and heterogeneous self- versus cross-attention layers. The authors therefore target three related problems: efficiency, flexibility across training phases, and practical debugging at scale.

What is the solution of this paper?

The central solution is a carefully co-designed **4D parallelism strategy: FSDP + TP + PP + CP**. For 8K-context 405B training, the authors use TP=8 and PP=16, keeping TP inside an eight-GPU NVLink domain while using PP rather than more expensive ZeRO-3 communication to distribute the model. For 131K context, they introduce CP=16 by replacing part of the DP dimension, keeping each GPU responsible for roughly 8K tokens.

The paper also extends interleaved 1F1B into a **flexible PP schedule** whose number of consecutive microbatches need not equal the number of pipeline ranks. Extra warm-up microbatches can hide P2P communication at the cost of additional activation memory. The authors further co-design the model partition itself: removing one Transformer layer from the first and last PP stages balances the expensive embedding/output modules. For long-context attention, they deliberately use a relatively simple **all-gather-based CP** rather than ring attention. Since K/V tensors are relatively small under GQA and attention computation grows quadratically with sequence length while communication grows linearly, exposed all-gather becomes relatively less important at long contexts and more easily supports irregular document masks.

How is the solution evaluated?

The evaluation combines controlled component studies with production measurements. Balanced PP reduces peak memory by about 5 GB and improves throughput by 6.5%; avoiding recomputation raises the improvement to 17.5% in the scaled-down experiment. CP reaches roughly 95% of single-GPU attention HFU at 128K sequences and performs comparably to or better than the ring-style TransformerEngine implementation in several configurations. End-to-end, Llama 3 405B reaches approximately **400 TFLOPs/GPU at 8K context and 380 TFLOPs/GPU at 131K context on 16K H100 GPUs**. The paper also evaluates debugging techniques for locating slow ranks and separating floating-point accumulation differences from actual implementation bugs.

Overall, what are 1–2 shortcomings of this paper?

First, much of the system is tightly co-designed with Llama 3, Meta's cluster topology, and internal software, which makes it difficult to determine how well every design choice transfers to other models and infrastructures. Second, its CP scheme deliberately uses static sequence sharding even for irregular document masks. In long-context production runs, the slowest GPU can take $1.44\times$ as long as the fastest because document boundaries create attention imbalance, showing that the system still leaves meaningful opportunities for dynamic load balancing.

Paper 2: *Tessera: A Holistic Pipeline Parallelism Framework for Trillion-Parameter Heterogeneous MoE Training*

What is the problem of this paper?

Existing pipeline systems generally assume stages consist of similar Transformer layers. Heterogeneous MoE architectures violate this assumption. Qwen3-Next mixes full softmax attention with Gated DeltaNet layers, followed by sparse MoE blocks; adjacent attention operations can differ by up to $10\times$ in compute time. Moreover, expert routing requires expensive All-to-All communication. Two pipeline partitions with equal serial execution time can therefore hide dramatically different amounts of communication—the paper measures overlap reductions ranging from 14.0% to 41.6% for different layer pairs. Finally, stochastic token routing changes expert workloads between iterations, producing bubbles that no static schedule can predict.

What is the solution of this paper?

Tessera treats **partitioning and communication overlap as a joint optimization problem**. First, it represents possible concurrent pipeline stages as an overlap graph. Candidate layer ranges around an initially balanced partition are decomposed into fine-grained task DAGs such as attention, dispatch, MLP, and combine. An event-driven overlap scheduler then constructs a specialized compute/communication interleaving for each candidate pair and profiles its actual **post-overlap cost** on hardware. A MILP uses those measured costs to select the partition minimizing the bottleneck stage rather than balancing serial layer time.

Tessera then handles unpredictable variation using a **Dynamic Bubble Optimizer (DBO)**. It distinguishes critical-path “backbone” work from movable tasks such as weight-gradient computation. Runtime routing metadata is used to identify likely idle slots, and movable work is deferred into those slots without altering model semantics or reduction order. This separates predictable structural heterogeneity, handled offline, from stochastic routing variation, handled online.

How is the solution evaluated?

The strongest evidence comes from production Qwen3 and Qwen3-Next training on **4,096–12,288 Hopper GPUs**. Across five workloads, Tessera improves throughput by **20%–33%**, including 39.0% MFU for a trillion-parameter Qwen3 workload. On a reproducible 256-GPU comparison using Qwen3-235B, DeepSeek-V3, and Nemotron-3 Super, it improves MFU over Alibaba's internal Megatron baseline by $1.27\times$, $1.24\times$, and $1.13\times$ respectively, and reaches up to $1.24\times$ the MFU of public Megatron-Core MoE recipes. Ablations also support the individual mechanisms: its heuristic overlap scheduler stays within roughly 1% of much slower ILP scheduling, while DBO reduces iteration time by 4.4%–5.4%, although it raises peak memory by roughly 2–4 percentage points.

Overall, what are 1–2 shortcomings of this paper?

First, Tessera depends on offline hardware profiling and searches only a bounded neighborhood around an initial partition. This keeps planning practical, specifically profiling remains under about an hour in the reported trillion-scale case, but can make plans hardware- and workload-specific and does not guarantee the global optimum. Second, overlap cannot manufacture computation where none exists. On a less compute-intensive Qwen3-Next-M workload, only 26% of EP communication is hidden and total exposed EP communication reaches 38.9% of iteration time. Thus Tessera is most effective when the compute-to-communication ratio provides enough useful work to cover communication.

3. Lecture Discussions

Question/Topic 1: Explain the all gather operation: previously ring attention - not all gather so all of the GPUs can reference entire context given to all GPUs, innovation: pad documents not in use with 0, so they save the slots in the context for the other documents while ensuring they don't pollute the context

Question/Topic 2: What is all to all: when computation requests are sent to various MoE shards, how is it different from all gather: in all-to-all, select data is sent to the GPUs hosting the experts required, in all gather, every GPU receives every piece of data to reconstruct the entire dataset

Question/Topic 3: Why is there forward AND backward overlap patterns: Forward pass is usually communication heavy and backward pass is calculating gradients and computation heavy, can't overlap compute on compute, can only overlap different functions

Question/Topic 4: If the overlapping scheduler can calculate such a good schedule, why do we still need the dynamic bubble optimization at runtime: MoE routing can take longer than anticipated which can allow for further optimization at runtime

Question/Topic 5: What does Relative FHU% mean on the Y axis of the chart: It may be a typo for HFU, Hardware Flop Utilization.

Question/Topic 6: You are saying that Tessera has a larger peak memory overhead. Why is this the case? Claims that more in-flight work, more overlap leads to more activations and task inputs kept in memory.

Question/Topic 7: It seems like Tessera on the right column is reducing memory. Where does the overhead in memory come from if this is the case? Tessera is said to reduce the train time in cost for more memory but it does not seem like the case in the chart: Because of DBO - it did decrease because it denotes that the maximum memory utilization decreased with DBO

Question/Topic 8: Both of the papers mention different modes. How generalizable are these approaches: It is less about the specific model and more about the architecture type. There are two types: homogeneous and heterogeneous. The architecture types and uniform layers are key to tessera's improvements.

Question/Topic 9: Why only test on Qwen3? Use it as a control, state of the art, MoE model with heterogeneity.

Question/Topic 10: Why did Qwen3 use MoE and Llama3 didn't: Llama3 is from 2024 and Qwen is a newer model - heterogeneity of model layers such as MoE (differing internal structures of different sizes) is a newer concept

Question/Topic 11: If you wanted to apply the heterogeneity to other models, how can you do this: It has to do with the internal structure of the model and due to the different types of attention mechanisms. By introducing mixed attention methods or different methodologies with different internal structures, you can have heterogeneity. A model is either heterogeneous or homogeneous.

Question/Topic 12: Why don't all models have heterogeneity (?): because most models (except for maybe DeepSeek that's focused on lowest cost) simply want the highest accuracy possible and are not very worried about potential cost saving resulting from heterogeneity

Question/Topic 13: What would be the recommendation from the Tessera folks about what would be the next most useful hardware innovation for the large scale heterogeneous training: CPU would be faster but they require a much different network to facilitate the communication - it's more important to have more homogeneous hardware to enable better scheduling predictions, hardware constantly changes ex.: Nvidia buying Groq for Vera Rubin decoders, hard to say that you build perfect hardware which is why software exists, only 42-45% FLOP utilization when utilized maximally - plenty of room for software to improve utilization, don't necessarily need better hardware

Question/Topic 14: To what extent is the necessity of advancements in parallelism research? Building something big, having smaller pieces to tolerate some failures, needs distribution and parallelism to move things around (discussed very generally).

Question/Topic 15: They mention an optimal performance but in real life there is not a ground truth as some hardware are stragglers and may fall behind. In MOE, there are different routing which can cause there to not be one ground truth as well as different routing can lead to differing performance. It seems that saying there is an optimal performance is risky as runtime variability is great: We have heuristics that we use to find the theoretical optimal, though actually calculating optimal performance is very hard. Both the papers have contributions of static and runtime improvements for pre-training. In llama 3, we look at the issues of irregular document length in tessera, we are looking at dynamic consideration of movable tasks so we can claim there are improvements even while true optimal is unknown.

Question/Topic 16: When using context parallelism, aren't they solving the problem they created in the first place of using long context: usually long context runs into a data issue on 128k token context not to create a problem for themselves and then solve it but it wouldn't fit on a single GPU, data parallelism is different because it's putting batches of contexts on different GPUs, but this is splitting the actual contexts between machines because they're too large

4. Proposal for Follow-Up Work

A remaining problem is that Tessera's pipeline **partition is still chosen offline**. DBO adapts the placement of movable tasks to per-iteration routing variation, but it does not alter stage boundaries. Over a long pre-training run, however, sequence lengths, routing distributions, model phases, or hardware behavior may change enough that the originally profiled post-overlap costs no longer describe the workload. The optimal static partition could therefore gradually become suboptimal.

I propose an **online routing-aware pipeline reconfiguration system**. Offline, the system would generate several high-quality Tessera-style partition/schedule candidates rather than one plan. During training, inexpensive telemetry, stage latency, A2A exposure, expert token distributions, and bubble time, would estimate which candidate best matches the current operating regime. If the predicted gain exceeds the one-time migration cost, the system would switch partitions at a safe optimizer/checkpoint boundary. A hysteresis threshold would prevent oscillation, while a memory constraint would rule out plans incompatible with the current activation footprint.

Evaluation should compare this system against Megatron-Core, static Tessera, and Tessera+DBO on both stable workloads and controlled distribution shifts. The main metrics would be MFU, mean and p95 iteration time, exposed communication, repartitioning overhead, and additional memory. Experiments should vary sequence length, MoE routing skew, and injected accelerator/network slowdowns. Correctness should additionally require identical convergence, or deterministic bitwise equivalence where possible, before any throughput improvement is considered successful.

5. Reflection on AI Usage

We stated that we thought efficient distributed training of very large language models was pointless compared to model performance, and we asked “how I should change my point of view?”. GPT-5.6 Sol told us to change our initial framing to “this field is easy to underestimate because most of its wins are invisible when it works.” It made a pretty good point that if you were to mostly care about model architecture, learning algorithms, or capabilities, distributed-training can feel like plumbing, such as PP schedules, all-gathers, routing imbalance, network topology, etc. However, especially as Kevin keeps mentioning, these 1 million dollar jobs, figuring the “plumbing” could lead to mitigating small percentage losses that can add up to an enormous amount of compute which is the money making part.

We asked questions about specific figures and images in the papers like Llama3 pipeline schedules and memory/throughout graphs to see if it could explain images to us that weren't straight up regurgitating the text in the paper.

One of our main points of confusion was how continuously smaller microbatches and more splitting data between GPUs could yield productivity gains forever. At some point, doesn't the amount of data become so small that the computation speed gains from splitting the work are less than the cost of the communication? We asked ChatGPT, and it helped highlight how the papers specifically and subtly address this limitation.